OIC Fault Handling Deep Dive : Assign vs Data Stitch, Scope Fault Handler and getFaultAsString() Behavior

Integration Overview

Purpose: This integration was designed as a learning and demonstration lab to explore and compare the behavior of the Assign action, the Data Stitch action, and OIC fault handling mechanisms including the Scope Fault Handler, getFaultAsString(), and Throw New Fault.

The integration accepts a Purchase Order number as input, invokes an external SOAP service (Oracle BI Publisher) to generate a PO report.

The integration is an App-Driven Orchestration triggered by a REST GET request. It is structured in four main steps.

Step 1 — Variable Initialization

Before entering the scope, an Assign action named AssignGlobalVars initializes the local variable **vErrorMsg** to an empty string. This ensures the variable exists and is accessible throughout the entire integration flow.

Step 2 — Scope : scopePOReport (Main Flow)

Inside the scope, a SOAP Invoke action named RunPOReportSOAP calls an external Oracle BI Publisher service to generate a PO report. The response is then evaluated by a Switch action with two branches.

The first branch, **Route 1**, is triggered when the length of the returned report bytes is greater than or equal to 20 characters, meaning a valid report was returned. In this case the flow continues normally with a decode and a stage file action.

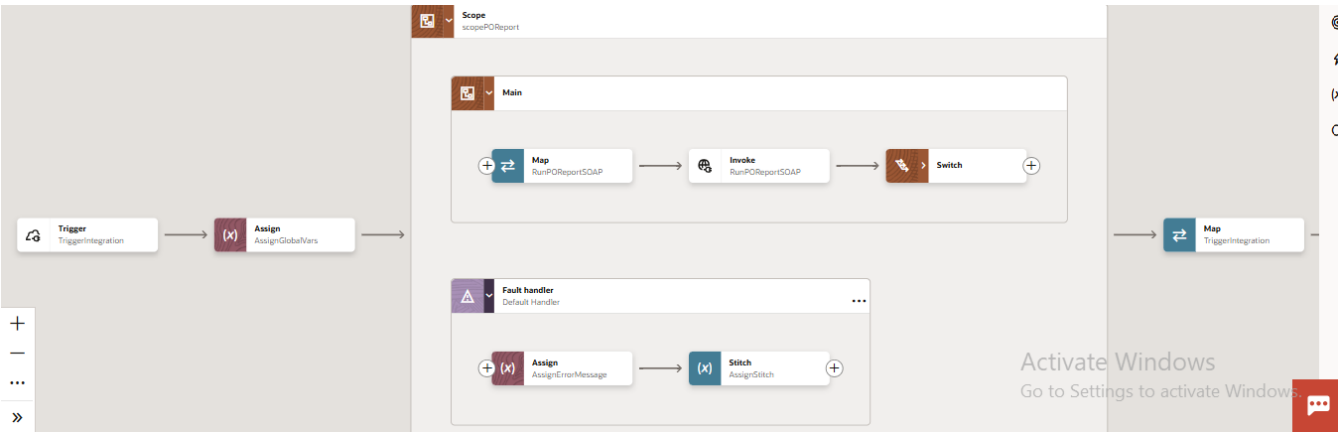
The second branch, **Otherwise**, is triggered when the PO number was not found in the SaaS system. In this branch, a Stitch action named Stitch2 directly assigns a static business error message to the global variable **gvErrorMSG**.

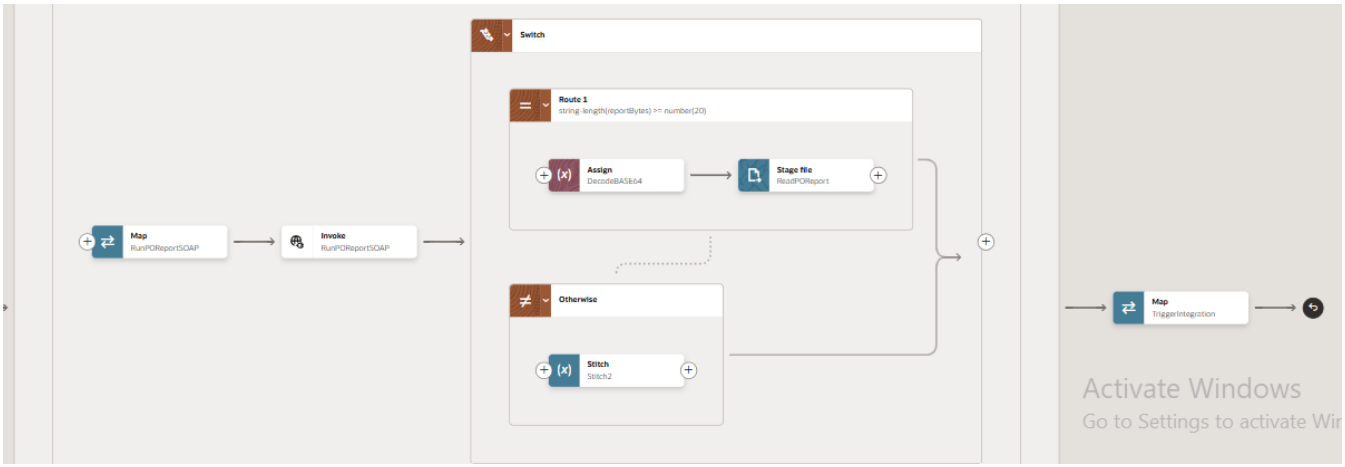
Step 3 — Scope : scopePOReport (Fault Handler)

The Default Fault Handler is activated only when a real system fault occurs inside the scope — either a remote SOAP fault from the external service, or a fault explicitly raised by a Throw New Fault action. It contains two actions running in sequence. First, an Assign action named AssignErrorMessage captures the full fault details into the local variable vErrorMsg using the getFaultAsString() function. Second, a Stitch action named AssignStitch captures the same fault details into the global variable gvErrorMSG using the same getFaultAsString() function.

Step 4 — Response Mapper and REST Response

After the scope completes — whether through the main flow or the fault handler — a Response Mapper maps both variables to the REST response payload. **vErrorMsg** is mapped to the field **ErrorMsgAssign** and **gvErrorMSG** is mapped to the field **ErrorMsgStitch**. The populated response is then returned to the caller.





##

Test 1 — System Fault : Invalid Report Path

Scenario: The SOAP invoke is triggered with an incorrect report path, causing the external BI Publisher service to return a SOAP fault.

<u>What happens:</u>

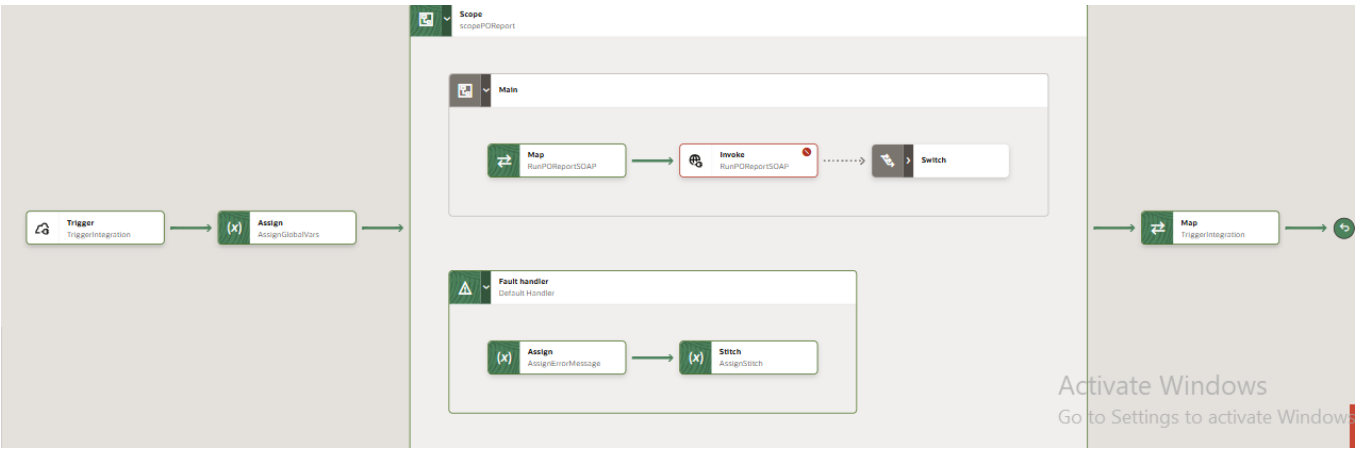
Invoke SOAP → external service returns SOAPFault

→ Scope Fault Handler is triggered

→ Assign : vErrorMsg = getFaultAsString()

→ Stitch : gvErrorMSG = getFaultAsString()

→ Both variables contain the full fault details



Response:

{

"ErrorMsgAssign": "faultName : {http://schemas.oracle.com/bpel/intension}remoteFault

Fault String : {http://schemas.oracle.com/bpel/intension}remoteFault

Fault Code : {http://schemas.oracle.com/bpel/intension}remoteFault

reason : Received a SOAPFault while invoking https://url:443/xmlpserver/services/internalReportWSSService due to \"PublicReportService::generateReport for reportAbsolutePath [/Custom/FZ/PO Reportxdo] Failure: Due to oracle.xdo.servlet.CreateException: oracle.xdo.servlet.CreateException: Report definition not found:/Custom/FZ/PO Reportxdo [username=user-int@****]\"

traceId : **********

requestId : 1XDIW3RCX.../FEIE7EW4Z.../L4M5IXQY...

errorId : ********

flowId : ********

origin : tech-adapters-****

errorCode : CASDK-0033

FaultDetails :

<details xmlns=\"http://xmlns.oracle.com/cloud/oic/gen3fault\">

<details xmlns:ns2=\"http://xmlns.oracle.com/cloud/oic/gen3fault\" xmlns=\"\">CASDK-0033: Received a SOAP fault while invoking endpoint target: https://url:443/xmlpserver/services/internalReportWSSService. This indicates a processing exception on the service endpoint side. Please check service side logs to further diagnose the problem

Fault Code : env:Receiver

Fault Reason :

PublicReportService::generateReport for reportAbsolutePath [/Custom/FZ/PO Reportxdo] Failure: Due to oracle.xdo.servlet.CreateException: oracle.xdo.servlet.CreateException: Report definition not found:/Custom/FZ/PO Reportxdo [username=user-int@****]

</details>

</details>",

"ErrorMsgStitch": "faultName : {http://schemas.oracle.com/bpel/intension}**remoteFault**

Fault String : {http://schemas.oracle.com/bpel/intension}**remoteFault**

Fault Code : {http://schemas.oracle.com/bpel/intension}**remoteFault**

reason : Received a SOAPFault while invoking https://url:443/xmlpserver/services/internalReportWSSService due to \"PublicReportService::generateReport for reportAbsolutePath [/Custom/FZ/PO Reportxdo] Failure: Due to oracle.xdo.servlet.CreateException: oracle.xdo.servlet.CreateException: Report definition not found:/Custom/FZ/PO Reportxdo [username=user-int@****]\"

traceId : **********

requestId : 1XDIW3RCX.../FEIE7EW4Z.../L4M5IXQY...

errorId : ********

flowId : ********

origin : tech-adapters-****

errorCode : CASDK-0033

FaultDetails :

<details xmlns=\"http://xmlns.oracle.com/cloud/oic/gen3fault\">

<details xmlns:ns2=\"http://xmlns.oracle.com/cloud/oic/gen3fault\" xmlns=\"\">CASDK-0033: Received a SOAP fault while invoking endpoint target: https://url:443/xmlpserver/services/internalReportWSSService. This indicates a processing exception on the service endpoint side. Please check service side logs to further diagnose the problem

Fault Code : env:Receiver

Fault Reason :

PublicReportService::generateReport for reportAbsolutePath [/Custom/FZ/PO Reportxdo] Failure: Due to oracle.xdo.servlet.CreateException: oracle.xdo.servlet.CreateException: Report definition not found:/Custom/FZ/PO Reportxdo [username=user-int@****]

</details>

</details>"

}

Key Observations:

Both Assign and Stitch return identical values in this scenario

getFaultAsString() returns a rich structured string containing: faultName, errorCode, reason, traceId, requestId, errorId, flowId, origin, FaultDetails

The fault type is **remoteFault** — an OIC/BPEL inherited wrapper for any fault crossing a service boundary

errorCode: CASDK-0033 identifies this as a system-level OIC technical error

Test 2 — Business Fault: Stitch in Otherwise Branch

Scenario: The report path is corrected. The SOAP invoke succeeds technically, but the report bytes returned are too short (PO number does not exist in SaaS). The Switch routes to the Otherwise branch where a **Stitch2** directly assigns a static message to gvErrorMSG

What happens:

Invoke SOAP → succeeds technically

→ Switch evaluates : string-length(reportBytes) >= 20 → FALSE

→ Otherwise branch executed

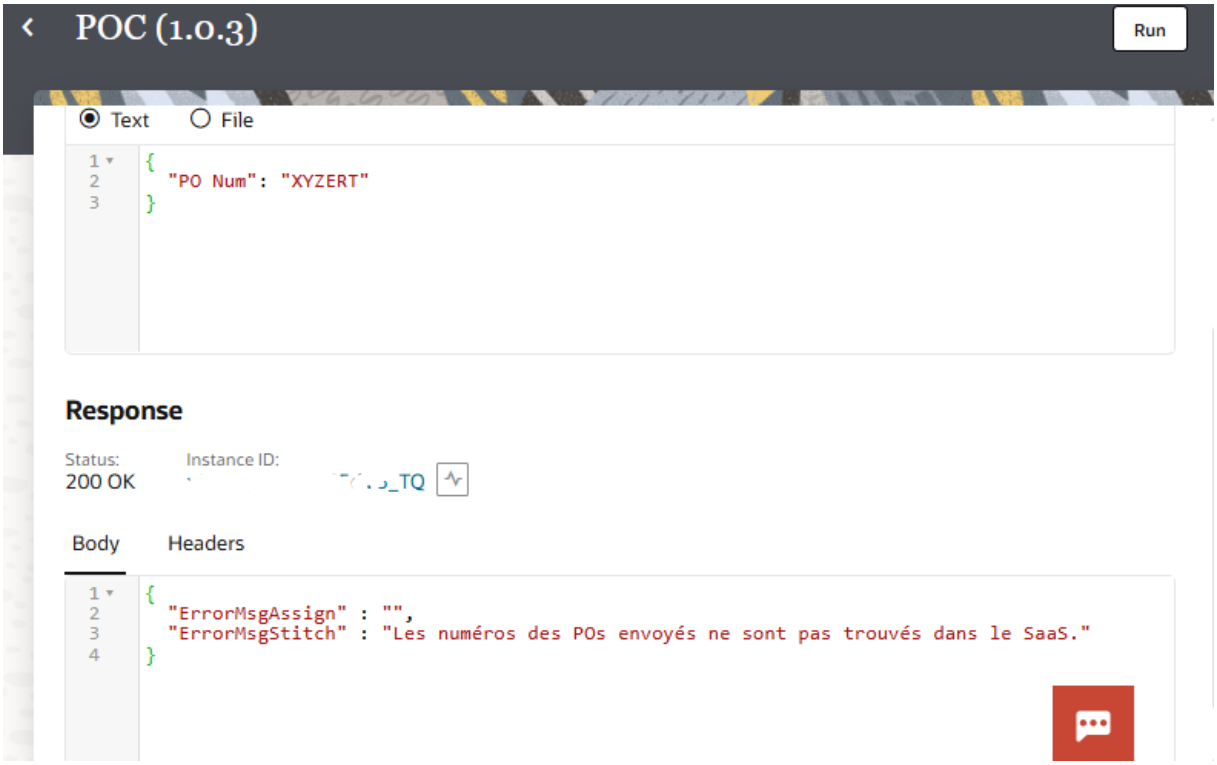
→ Stitch2 : gvErrorMSG = 'Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.'

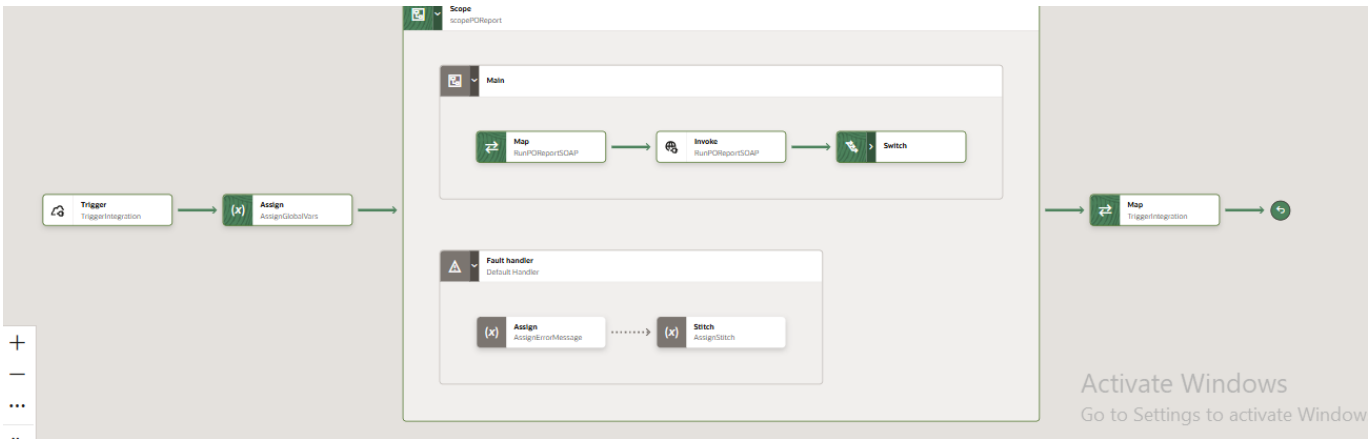
→ NO fault raised → Scope Fault Handler is NOT triggered

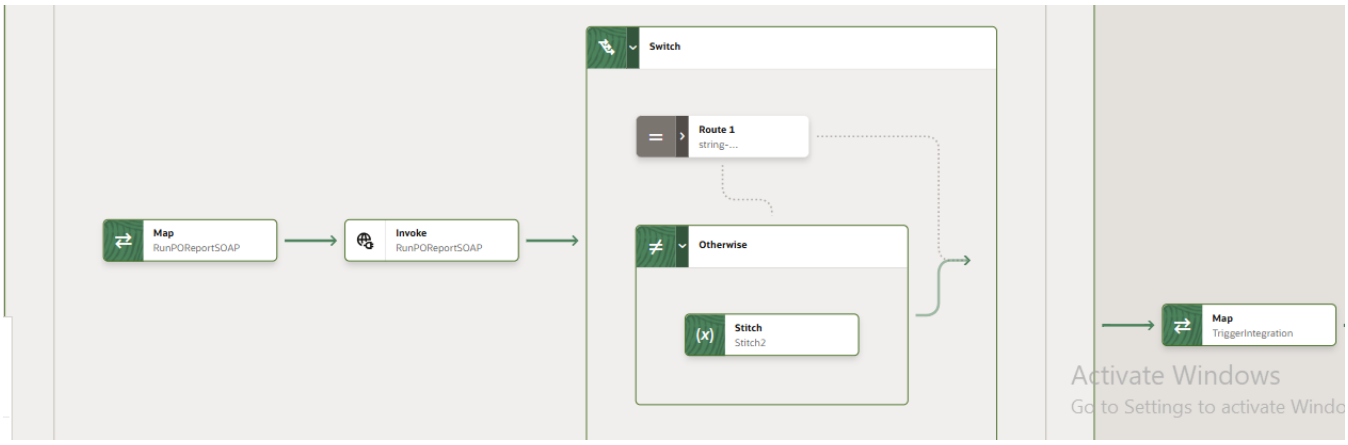
→ Assign in Fault Handler : NEVER executed → vErrorMsg stays ''

→ Stitch in Fault Handler : NEVER executed

Otherwise
Stitch
Stitch2
Configure stitch
Stitch2
Specify as many variables for this stitch as you need, and configure their values.
gvErrorMSG
=
'Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.'
Activate Windows


POC (1.0.3)
Run
Text
File
{
"PO Num": "XYZERT"
}
Response
Status:
200 OK
Instance ID:
Body
Headers
{
"ErrorMsgAssign" : "",
"ErrorMsgStitch" : "Les numéros des POs envoyés ne sont pas trouvés dans le SaaS."
}


Scope
scopePOReport
Main
Map
RunPOReportSOAP
Invoke
RunPOReportSOAP
Switch
Trigger
TriggerIntegration
Assign
AssignGlobalVars
Fault handler
Default Handler
Assign
AssignErrorMessage
Stitch
AssignStitch
Map
TriggerIntegration
Activate Windows
Go to Settings to activate Window



Key Observations:

ErrorMsgAssign is empty — the Assign inside the Fault Handler never ran

ErrorMsgStitch is populated — because gvErrorMSG is a Global Variable, modified directly in the Otherwise branch via Stitch2

This proves the fundamental difference between a local variable and a Global Variable in OIC

Test 3 — Business Fault : Throw New Fault in Otherwise Branch

Scenario: Same as Test 2, but we remove Stitch 2. Instead, the Switch routes to the Otherwise branch, which triggers a Throw New Fault.

What happens:

Invoke SOAP → succeeds technically

→ Switch evaluates : string-length(reportBytes) >= 20 → FALSE

→ Otherwise branch executed

→ Throw New Fault :

Code : 404

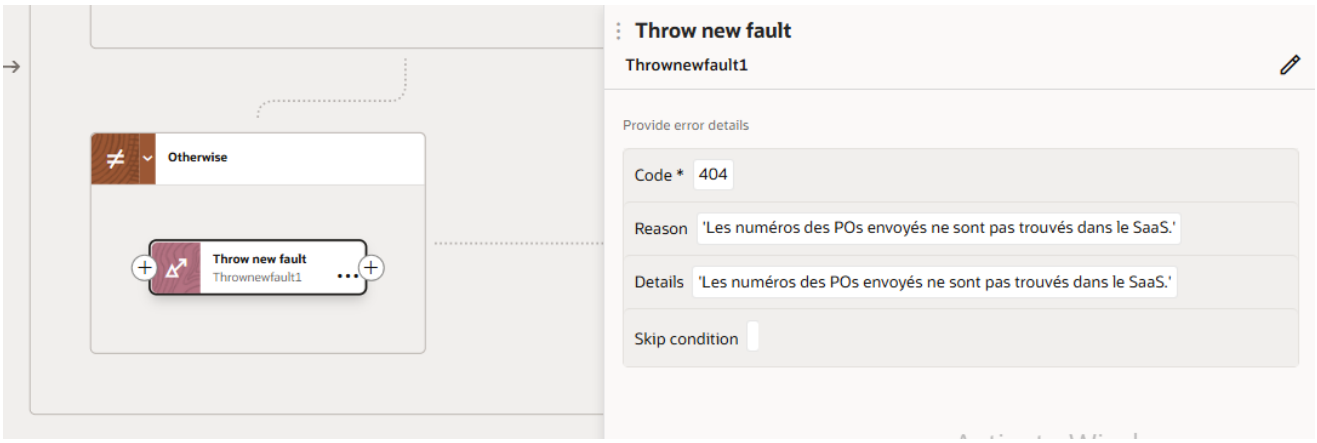
Reason : 'Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.'

Details: 'Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.'

→ Throw New Fault triggers the Scope Fault Handler

→ Assign : vErrorMsg = getFaultAsString()

→ Stitch : gvErrorMSG = getFaultAsString()



Response:

{

"ErrorMsgAssign": "faultName : {http://schemas.oracle.com/bpel/extension}remoteFault

Fault String : {http://schemas.oracle.com/bpel/extension}remoteFault

Fault Code : {http://schemas.oracle.com/bpel/extension}remoteFault

reason : \"Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.\"

traceId : **********

requestId : AEFO5J6E1.../WLUXFFGAM.../7HJA1YX6R...

errorId : ********

flowId : ********

origin : mcube

errorCode : 404.0

FaultDetails :

<t xmlns=\"\">Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.</t>",

"ErrorMsgStitch": "faultName : {http://schemas.oracle.com/bpel/extension}remoteFault

Fault String : {http://schemas.oracle.com/bpel/extension}remoteFault

Fault Code : {http://schemas.oracle.com/bpel/extension}remoteFault

reason : \"Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.\"

traceId : **********

requestId : AEFO5J6E1.../WLUXFFGAM.../7HJA1YX6R...

errorId : ********

flowId : ********

origin : mcube

errorCode : 404.0

FaultDetails :

<t xmlns=\"\">Les numéros des POs envoyés ne sont pas trouvés dans le SaaS.</t>"

}

Key Observations:

Even a custom Throw New Fault is wrapped by OIC as remoteFault — this is expected BPEL engine behavior

The errorCode is now 404.0 (your custom code) — this is what differentiates a business fault from a system fault, not the faultName

FaultDetails contains exactly what you put in the Details field of Throw New Fault

Critical insight: faultName alone cannot distinguish system errors from business errors you must parse errorCode inside getFaultAsString()

##

Conclusion :

The recommended pattern combines three elements. First, declare a **Global Variable** to store the error message so it remains accessible anywhere in the flow regardless of scope boundaries. Second, always add a **Throw New Fault** at the end of the Otherwise branch — this is the key step that forces any business condition error to go through the Fault Handler, just like a real system fault would. Third, inside the Fault Handler, use a **Data Stitch action** to capture the full error context into the Global Variable using getFaultAsString(). This way, whether the error is a system fault or a business condition, both paths are unified through the Fault Handler, and the Global Variable always contains the complete error details when the Response Mapper runs.